

SQL Tuning Case Study

Solution : Join

Join Tuning

JOIN CASE 1

다음 문장의 실행 계획을 확인하고 Join의 성능을 최적화 시킨다.

```
SQL>SELECT      c.cust_id,
                c.cust_last_name,
                c.cust_city,
                c.cust_credit_limit,
                s.prod_id,
                s.time_id,
                s.quantity_sold
FROM            custs c, sales s
WHERE           c.cust_id = s.cust_id
AND             c.cust_city = 'Los Angeles'
AND             c.cust_credit_limit > 3000
AND             s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
AND             TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1932	00:00:00.02	9688
1	NESTED LOOPS		1		1932	00:00:00.02	9688
2	NESTED LOOPS		1	2580	8025	00:00:00.01	2371
* 3	TABLE ACCESS FULL	CUSTS	1	80	629	00:00:00.01	1158
* 4	INDEX RANGE SCAN	SALES_IX02	629	129	8025	00:00:00.01	1213
* 5	TABLE ACCESS BY INDEX ROWID	SALES	8025	32	1932	00:00:00.01	7317

Predicate Information (identified by operation id):

```
3 - filter(("C"."CUST_CITY"='Los Angeles' AND "C"."CUST_CREDIT_LIMIT">3000))
4 - access("C"."CUST_ID"="S"."CUST_ID")
5 - filter(("S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyy-mm-dd hh24:mi:ss') AND
         "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'syyy-mm-dd hh24:mi:ss')))
```

> 답안. Nested Loops Join의 최적화 방법

1. Looping을 최소화시킬 수 있도록 Driving Table을 선택한다.
2. Inner Table에는 조인 컬럼을 선행 컬럼으로 포함한 결합 인덱스를 준비한다.
3. 최대한 인덱스에서 모든 조건을 비교 가능하게 하여 테이블에 대한 Random Access를 줄인다.

```
SQL> @idx
Enter value for tab_name: custs
```

INDEX_NAME	INDEX_TYPE	UNIQUENESS	COLUMNS
CUST_IX01	NORMAL	UNIQUE	CUST_ID

```
SQL> create index custs_new on custs(cust_city) nologging;
SQL> create index sales_new on sales(cust_id, time_id) nologging;
SQL> SELECT      /*+ use_nl(s c) index(c custs_new) */ c.cust_id,
                c.cust_last_name,
                c.cust_city,
                c.cust_credit_limit,
                s.prod_id,
                s.time_id,
                s.quantity_sold
FROM            custs c, sales s
WHERE           c.cust_id = s.cust_id
AND             c.cust_city = 'Los Angeles'
AND             c.cust_credit_limit > 3000
AND             s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
AND             TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1932	00:00:00.01	3684
1	NESTED LOOPS		1		1932	00:00:00.01	3684
2	NESTED LOOPS		1	2580	1932	00:00:00.01	1830
* 3	TABLE ACCESS BY INDEX ROWID	CUSTS	1	80	629	00:00:00.01	623
* 4	INDEX RANGE SCAN	CUSTS_NEW	1	90	932	00:00:00.01	25
* 5	INDEX RANGE SCAN	SALES_NEW	629	32	1932	00:00:00.01	1207
6	TABLE ACCESS BY INDEX ROWID	SALES	1932	32	1932	00:00:00.01	1854

```
Predicate Information (identified by operation id):
```

```
3 - filter("C"."CUST_CREDIT_LIMIT">3000)
4 - access("C"."CUST_CITY"='Los Angeles')
5 - access("C"."CUST_ID"="S"."CUST_ID" AND "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00',
'syyyy-mm-dd hh24:mi:ss') AND "S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyyy-mm-dd
hh24:mi:ss'))
```

```
SQL> drop index custs_new;
SQL> drop index sales_new;
```

JOIN CASE 2

다음 문장의 실행 계획을 확인하고 성능을 최적화 시킨다.

```
SQL>SELECT      /*+ leading(c) use_nl(s) */ COUNT(*)
      FROM      sales s, custs c
      WHERE     s.cust_id = c.cust_id
      AND       c.country_id = 52790
      AND       s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
                        AND      TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:01.67	453K
1	SORT AGGREGATE		1	1	1	00:00:01.67	453K
2	NESTED LOOPS		1		140K	00:00:01.65	453K
3	NESTED LOOPS		1	94710	522K	00:00:00.37	37377
* 4	TABLE ACCESS FULL	CUSTS	1	2921	18520	00:00:00.01	1139
* 5	INDEX RANGE SCAN	SALES_IX02	18520	129	522K	00:00:00.22	36238
* 6	TABLE ACCESS BY INDEX ROWID	SALES	522K	32	140K	00:00:01.04	416K

Predicate Information (identified by operation id):

```
4 - filter("C"."COUNTRY_ID"=52790)
5 - access("S"."CUST_ID"="C"."CUST_ID")
6 - filter(("S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyy-mm-dd hh24:mi:ss') AND
        "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'syyy-mm-dd hh24:mi:ss')))
```

> 답안 1. Nested Loop Join 기법에서 인덱스의 최적화 (Case 2와 같이 해결)

```
SQL> create index custs_new on custs(country_id) nologging;
SQL> create index sales_new on sales(cust_id, time_id) nologging;
SQL> SELECT      /*+ leading(c) use_nl(s) */ COUNT(*)
      FROM        sales s, custs c
      WHERE       s.cust_id = c.cust_id
      AND         c.country_id = 52790
      AND         s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
      AND         TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:00.14	10212
1	SORT AGGREGATE		1	1	1	00:00:00.14	10212
2	NESTED LOOPS		1	94710	140K	00:00:00.14	10212
* 3	VIEW	index\$_join\$_002	1	2921	18520	00:00:00.06	165
* 4	HASH JOIN		1		18520	00:00:00.05	165
* 5	INDEX RANGE SCAN	CUSTS_NEW	1	2921	18520	00:00:00.01	43
6	INDEX FAST FULL SCAN	CUST_IX01	1	2921	55500	00:00:00.01	122
* 7	INDEX RANGE SCAN	SALES_NEW	18520	32	140K	00:00:00.05	10047

Predicate Information (identified by operation id):

```
3 - filter("C"."COUNTRY_ID"=52790)
4 - access(ROWID=ROWID)
5 - access("C"."COUNTRY_ID"=52790)
7 - access("S"."CUST_ID"="C"."CUST_ID" AND "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00',
'syyyy-mm-dd hh24:mi:ss') AND "S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyyy-mm-dd
hh24:mi:ss'))
```

처음보다는 나아졌으나 LOOP 횟수가 많고 조인 결과 건수가 많음

Hash Join 으로 시도해보자

> 답안 2. Hash Join 사용

```
SQL> SELECT      /*+ leading(c) use_hash(s) */ COUNT(*)
FROM            sales s, custs c
WHERE           s.cust_id = c.cust_id
AND             c.country_id = 52790
AND             s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
AND             TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

```
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:00.33	3206
1	SORT AGGREGATE		1	1	1	00:00:00.33	3206
* 2	HASH JOIN		1	94710	140K	00:00:00.32	3206
* 3	VIEW	index\$_join\$_002	1	2921	18520	00:00:00.02	165
* 4	HASH JOIN		1		18520	00:00:00.02	165
* 5	INDEX RANGE SCAN	CUSTS_NEW	1	2921	18520	00:00:00.01	43
6	INDEX FAST FULL SCAN	CUST_IX01	1	2921	55500	00:00:00.01	122
* 7	INDEX FAST FULL SCAN	SALES_NEW	1	228K	246K	00:00:00.13	3041

```
Predicate Information (identified by operation id):
```

```
2 - access("S"."CUST_ID"="C"."CUST_ID")
3 - filter("C"."COUNTRY_ID"=52790)
4 - access(ROWID=ROWID)
5 - access("C"."COUNTRY_ID"=52790)
7 - filter(("S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyy-mm-dd hh24:mi:ss') AND
          "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'syyy-mm-dd hh24:mi:ss')))
```

> 답안 3. Hash Join의 최적화에 필요한 인덱스 사용

```
SQL> CREATE INDEX custs_x01 ON custs(country_id,cust_id) nologging ;
SQL> CREATE INDEX sales_x01 ON sales(time_id,cust_id) nologging ;
SQL> SELECT      /*+ leading(c) use_hash(s) */ COUNT(*)
  FROM          sales s, custs c
 WHERE          s.cust_id = c.cust_id
    AND         c.country_id = 52790
    AND         s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
    AND         TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:00.16	856
1	SORT AGGREGATE		1	1	1	00:00:00.16	856
* 2	HASH JOIN		1	94710	140K	00:00:00.15	856
* 3	INDEX RANGE SCAN	CUSTS_X01	1	2921	18520	00:00:00.01	56
* 4	INDEX RANGE SCAN	SALES_X01	1	228K	246K	00:00:00.03	800

```
Predicate Information (identified by operation id):
```

```
2 - access("S"."CUST_ID"="C"."CUST_ID")
3 - access("C"."COUNTRY_ID"=52790)
4 - access("S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'syyy-mm-dd
    hh24:mi:ss') AND "S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'syyy-mm-dd
    hh24:mi:ss'))
```

> 결론

Nested Loops Join 은 Looping 의 횟수와 Inner Table에 조건에 만족하는 행이 많아질수록 실행 시간은 늘어난다. 대량의 데이터를 조인할 경우에는 Hash Join을 사용하며 Hash Join에 최적화된 인덱스를 준비한다. Hash Join에서의 결합 인덱스는 Join 컬럼보다 조건식에 사용되는 컬럼이 선행 컬럼으로 사용되어야 Index Range Scan 이 가능하다.

```
SQL> DROP INDEX custs_new ;
SQL> DROP INDEX custs_x01 ;
SQL> DROP INDEX sales_new ;
SQL> DROP INDEX sales_x01 ;
```

JOIN CASE 3

다음 문장의 실행 계획을 확인하고 성능을 최적화 시킨다.

```
SQL> SELECT    DISTINCT c.channel_id, c.channel_desc, s.prod_id, s.promo_id
      FROM      channels c, sales s
      WHERE     c.channel_id = s.channel_id
      AND       s.prod_id BETWEEN 10 AND 20 ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		53	00:00:00.02	289			
1	HASH UNIQUE		1	142	53	00:00:00.02	289	1246K	1246K	1375K (0)
* 2	HASH JOIN		1	61703	61702	00:00:00.01	289	1506K	1506K	898K (0)
3	TABLE ACCESS FULL	CHANNELS	1	5	5	00:00:00.01	9			
* 4	INDEX RANGE SCAN	SALES_IX01	1	61703	61702	00:00:00.01	280			

Predicate Information (identified by operation id):

```
2 - access("C"."CHANNEL_ID"="S"."CHANNEL_ID")
4 - access("S"."PROD_ID">=10 AND "S"."PROD_ID"<=20)
```

문제점)

JOIN 작업 후 DISTINCT 작업 수행됨

두 테이블의 관계는 1:M 관계를 가지고 있으며 DISTINCT를 어디에서 사용하던 동일한 결과를 갖는 문장이다. JOIN 작업의 성능을 위해서라면 JOIN에 참여하는 행의 개수를 줄이는 것이 성능상 유리하다.

해결)

```
SQL> SELECT c.channel_id, c.channel_desc, s.prod_id, s.promo_id
FROM channels c,
      (select distinct channel_id, prod_id, promo_id
       from sales
       where prod_id between 10 and 20) s
WHERE c.channel_id = s.channel_id ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		53	00:00:00.01	288			
1	MERGE JOIN		1	40	53	00:00:00.01	288			
2	TABLE ACCESS BY INDEX ROWID	CHANNELS	1	5	5	00:00:00.01	8			
3	INDEX FULL SCAN	CHANNELS_IX01	1	5	5	00:00:00.01	4			
* 4	SORT JOIN		5	40	53	00:00:00.01	280	2048	2048	2048 (0)
5	VIEW		1	40	53	00:00:00.01	280			
6	HASH UNIQUE		1	40	53	00:00:00.01	280	1646K	1646K	1332K (0)
* 7	INDEX RANGE SCAN	SALES_IX01	1	61703	61702	00:00:00.01	280			

Predicate Information (identified by operation id):

```
4 - access("C"."CHANNEL_ID"="S"."CHANNEL_ID")
    filter("C"."CHANNEL_ID"="S"."CHANNEL_ID")
7 - access("PROD_ID">=10 AND "PROD_ID"<=20)
```

<- 조인에 참여하는 행의 건수가 줄어서 정렬에 사용하는 메모리 크기가 줄었음

JOIN CASE 4

PRODS, SALES 테이블을 이용하여 제품별 판매 수량(QUANTITY_SOLD)의 합계를 다음과 같이 검색하는 명령어를 작성하세요. 판매되지 않은 제품이 존재한다면 해당 제품도 함께 표시합니다. 실행 계획을 확인하고 성능을 최적화시킵니다.

검색결과

PROD_ID	PROD_NAME	SOLD_SUM
13	5MP Telephoto Digital Camera	6002
14	17" LCD w/built-in HDTV Tuner	5998
15	Envoy 256MB - 40GB	5766
16	Y Box	6929
17	Mini DV Camcorder with 3.5" Swivel LCD	6160
18	Envoy Ambassador	9591
19	Laptop carrying case	10430
20	Home Theatre Package with DVD-Audio/Video Play	10826
...		

72 행이 선택되었습니다.

> 답안 1. Outer Join 이용

```
SQL> SELECT
      p.prod_id, p.prod_name, SUM(s.quantity_sold) AS sold_sum
FROM    prods p, sales s
WHERE    p.prod_id = s.prod_id (+)
GROUP BY p.prod_id, p.prod_name ;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		72	00:00:01.16	4463			
1	HASH GROUP BY		1	72	72	00:00:01.16	4463	793K	793K	3004K (0)
2	HASH JOIN OUTER		1	913K	913K	00:00:00.84	4463	933K	933K	1242K (0)
3	TABLE ACCESS FULL	PRODS	1	72	72	00:00:00.01	2			
4	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:00.15	4461			

```
Predicate Information (identified by operation id):
```

```
2 - access("P"."PROD_ID"="S"."PROD_ID")
```

> 문제 사항

Join 작업과 Grouping 작업량이 동일하기 때문에 자원 소모가 크다.

> 답안 2. GROUPING 작업 후 Join 진행

```
SQL> SELECT      p.prod_id, p.prod_name, s.sold_sum
FROM            prods p,
      ( SELECT prod_id, SUM(quantity_sold) AS sold_sum
        FROM      sales
        GROUP BY  prod_id ) s
WHERE           p.prod_id = s.prod_id (+) ;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		72	00:00:00.34	4465			
1	MERGE JOIN OUTER		1	72	72	00:00:00.34	4465			
2	TABLE ACCESS BY INDEX ROWID	PRODS	1	72	72	00:00:00.01	4			
3	INDEX FULL SCAN	PROD_IX01	1	72	72	00:00:00.01	2			
4	SORT JOIN		72	71	71	00:00:00.34	4461	9216	9216	8192 (0)
5	VIEW		1	71	71	00:00:00.34	4461			
6	HASH GROUP BY		1	71	71	00:00:00.34	4461	941K	941K	1339K (0)
7	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:00.14	4461			

```
Predicate Information (identified by operation id):
```

```
4 - access("P"."PROD_ID"="S"."PROD_ID")
    filter("P"."PROD_ID"="S"."PROD_ID")
```

SALES 테이블의 Grouping을 통해 조인에 참여할 행의 수를 PRODS 테이블의 수와 동일하게 만들었습니다. I/O의 횟수는 동일하지만 Join의 수행 시간은 더 빠르게 진행 가능합니다.

참고)

```
SQL> SELECT
      p.prod_id, p.prod_name, SUM(s.quantity_sold) AS sold_sum
    FROM      prods p, sales s
    WHERE     p.prod_id = s.prod_id (+)
    GROUP BY  p.prod_id, p.prod_name
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		72	00:00:10.40	4465
1	HASH GROUP BY		1	72	72	00:00:10.40	4465
2	HASH JOIN OUTER		1	913K	913K	00:00:08.61	4465
3	TABLE ACCESS FULL	PRODS	1	72	72	00:00:00.01	3
4	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:01.73	4462

Predicate Information (identified by operation id):

```
2 - access("P"."PROD_ID"="S"."PROD_ID")
```

```
SQL> SELECT
      p.prod_id, p.prod_name, SUM(s.quantity_sold) AS sold_sum
    FROM      prods p, sales s
    WHERE     p.prod_id = s.prod_id
    GROUP BY  p.prod_id, p.prod_name
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		71	00:00:00.10	4464
1	HASH GROUP BY		1	71	71	00:00:00.10	4464
2	MERGE JOIN		1	71	71	00:00:00.10	4464
3	TABLE ACCESS BY INDEX ROWID	PRODS	1	72	72	00:00:00.01	2
4	INDEX FULL SCAN	PROD_IX01	1	72	72	00:00:00.01	1
5	SORT JOIN		72	71	71	00:00:00.10	4462
6	VIEW	VW_GBC_5	1	71	71	00:00:00.10	4462
7	HASH GROUP BY		1	71	71	00:00:00.10	4462
8	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:00.04	4462

Predicate Information (identified by operation id):

```
5 - access("P"."PROD_ID"="ITEM_1")
    filter("P"."PROD_ID"="ITEM_1")
```

<= 아우터 조인이 아닐 경우는 Optimizer가 In-Line View로 자동 변환해 줍니다.
따라서 Outer Join 시는 Inline View를 이용하여 작성하는 것이 필요합니다.

12c 환경에서는 Outer Join 으로 수행할 때도 자동 변환을 해줍니다.

```
SQL> SELECT
      p.prod_id, p.prod_name, SUM(s.quantity_sold) AS sold_sum
    FROM      prods p, sales s
    WHERE     p.prod_id = s.prod_id (+)
    GROUP BY  p.prod_id, p.prod_name;
```

SQL>@xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		72	00:00:00.06	4474			
1	MERGE JOIN OUTER		1	72	72	00:00:00.06	4474			
2	TABLE ACCESS BY INDEX ROWID	PRODS	1	72	72	00:00:00.01	12			
3	INDEX FULL SCAN	PROD_IX01	1	72	72	00:00:00.01	6			
* 4	SORT JOIN		72	71	71	00:00:00.06	4462	9216	9216	8192 (0)
5	VIEW	VW_GBC_5	1	71	71	00:00:00.06	4462			
6	HASH GROUP BY		1	71	71	00:00:00.06	4462	1200K	1200K	1328K (0)
7	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:00.02	4462			

Predicate Information (identified by operation id):

```
4 - access("P"."PROD_ID"="ITEM_1")
    filter("P"."PROD_ID"="ITEM_1")
```

<= 아우터 조인도 Optimizer가 InLine View로 Transformation

```
SQL> SELECT
      p.prod_id, p.prod_name, SUM(s.quantity_sold) AS sold_sum
    FROM      prods p, sales s
    WHERE     p.prod_id = s.prod_id
    GROUP BY  p.prod_id, p.prod_name
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		71	00:00:00.08	4474			
1	MERGE JOIN		1	71	71	00:00:00.08	4474			
2	TABLE ACCESS BY INDEX ROWID	PRODS	1	72	72	00:00:00.01	12			
3	INDEX FULL SCAN	PROD_IX01	1	72	72	00:00:00.01	6			
* 4	SORT JOIN		72	71	71	00:00:00.08	4462	9216	9216	8192 (0)
5	VIEW	VW_GBC_5	1	71	71	00:00:00.08	4462			
6	HASH GROUP BY		1	71	71	00:00:00.08	4462	1200K	1200K	1330K (0)
7	TABLE ACCESS FULL	SALES	1	913K	913K	00:00:00.02	4462			

Predicate Information (identified by operation id):

```
4 - access("P"."PROD_ID"="ITEM_1")
    filter("P"."PROD_ID"="ITEM_1")
```